

Algorytmy Równoległe - Laboratorium 6

Adam Staniszewski

4 grudnia 2024

1 Zasada działania algorytmów typu branch and bound

Branch and bound to metoda rozwiązywania problemów kombinatorycznych, która opiera się na przeszukiwaniu przestrzeni rozwiązań przy eliminacji części przestrzeni, które nie mogą zawierać lepszego rozwiązania.

Kluczowe elementy algorytmu:

- **Drzewo przeszukiwań**

- Każdy węzeł drzewa reprezentuje częściowe rozwiązanie (podzbiór miast, które komiwojażer odwiedził, oraz aktualny koszt podróży),
- Korzeń drzewa odpowiada sytuacji początkowej (bez odwiedzonych miast, z wyjątkiem miasta startowego).

- **Branch**

- Z każdego węzła generowane są nowe węzły, które reprezentują kolejne możliwe decyzje (odwiedzenie następnych miast),
- Dla każdego nowego węzła aktualizowany jest koszt trasy.

- **Bound**

- Algorytm oblicza dolne ograniczenie kosztu dla każdej gałęzi,
- Dolne ograniczenie jest oszacowaniem minimalnego możliwego kosztu kontynuowania trasy z danego węzła,
- Jeżeli dolne ograniczenie jest wyższe niż koszt najlepszego znalezionego dotąd rozwiązania, gałąź jest odrzucana.

- **Pruning**

Jeśli dolne ograniczenie dla danego węzła jest większe niż koszt obecnego najlepszego rozwiązania, algorytm nie przeszukuje dalej tej gałęzi,

Algorytm kontynuuje rozgałęzianie i ograniczanie, aż wszystkie gałęzie zostaną albo rozwinięte, albo przycięte.

2 Kod

2.1 Stałe

```
MAXSIZE = float("inf")
N = 12
DEPTH = 2
AGGLOMERATION_RADIUS = 10
NOISE_LEVEL = 100
GENERATE_INSIDE = True
```

2.2 Generowanie miast

```
def generate_cities(
    n: int,
    radius: float = 10,
    noise_level: float = 42,
    generate_inside: bool = False
) -> list[tuple[float, float]]:
    angles = np.linspace(0, 2 * np.pi, n, endpoint=False)
    angles += np.random.uniform(-noise_level, noise_level, size=n)

    if generate_inside:
        radii = np.random.uniform(0, radius, size=n)
    else:
        radii = np.full(n, radius)

    return [(radii[i] * np.cos(angles[i]),
              radii[i] * np.sin(angles[i])) for i in range(n)]
```

2.3 Generowanie macierzy odległości

```
def distance_matrix(cities: list[tuple[float, float]]):
    N = len(cities)
    dist_mat: np.ndarray = np.zeros((N, N))

    for i in range(N):
        for j in range(N):
            dist_mat[i][j] = math.sqrt(
                (cities[i][0] - cities[j][0])**2 +
                (cities[i][1] - cities[j][1])**2
            )

    return dist_mat
```

2.4 Kopiowanie ścieżek

```
def copy_to_final(curr_path, final_path, N):
    final_path[:N + 1] = curr_path[:]
    final_path[N] = curr_path[0]
```

2.5 Pierwsze minimum

```
def first_min(adj: list[tuple[float, float]], i: int) -> float:
    min = MAXSIZE
    for j in range(len(adj)):
        if adj[i][j] < min and i != j:
            min = adj[i][j]
    return min
```

2.6 Drugie minimum

```
def second_min(adj: list[tuple[float, float]], i: int) -> float:
    first = second = MAXSIZE
    for j in range(len(adj)):
        if i == j:
            continue
        if adj[i][j] <= first:
            second = first
            first = adj[i][j]
        elif adj[i][j] <= second and adj[i][j] != first:
            second = adj[i][j]
    return second
```

2.7 Rekursywne TSP

```
def tsp_recursive(
    adj,
    curr_bound,
    curr_weight,
    level,
    curr_path,
    visited,
    final_path,
    N,
    final_res
):
    if level == N:
```

```

        if adj[curr_path[level - 1]][curr_path[0]] != 0:
            curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]]
            if curr_res < final_res[0]:
                copy_to_final(curr_path, final_path, N)
                final_res[0] = curr_res
        return

for i in range(N):
    if adj[curr_path[level-1]][i] != 0 and not visited[i]:
        curr_weight += adj[curr_path[level - 1]][i]
        if level == 1:
            curr_bound -= ((first_min(adj, curr_path[level - 1])) +
                           first_min(adj, i)) / 2
        else:
            curr_bound -= ((second_min(adj, curr_path[level - 1])) +
                           first_min(adj, i)) / 2

        if curr_bound + curr_weight < final_res[0]:
            visited[i] = True
            curr_path[level] = i
            tsp_recursive(
                adj,
                curr_bound,
                curr_weight,
                level + 1,
                curr_path,
                visited,
                final_path,
                N,
                final_res
            )
            visited[i] = False

        curr_weight -= adj[curr_path[level - 1]][i]
        curr_bound += ((second_min(adj, curr_path[level - 1])) +
                      first_min(adj, i)) / 2

```

2.8 Główna metoda wywołująca branch and bound

```

def branch_and_bound_tsp(dist_mat, N):
    final_res = [MAXSIZE]
    curr_bound = 0
    curr_path = [-1] * (N + 1)
    visited = [False] * N

```

```

for i in range(N):
    curr_bound += (first_min(dist_mat, i) + second_min(dist_mat, i))
    curr_bound = math.ceil(curr_bound / 2)

    curr_path[0] = 0
    visited[0] = True

    final_path = [-1] * (N + 1)
    tsp_recursive(
        dist_mat,
        curr_bound,
        0,
        1,
        curr_path,
        visited,
        final_path,
        N,
        final_res
    )

return final_res[0], final_path

```

3 Implementacja równoległa

```

def branch_and_bound_tsp_parallel(
    cities: list[tuple[float, float]],
    N: int,
    depth: int
):
    dist_mat = distance_matrix(cities)
    initial_paths = [[0, i] for i in range(1, min(N, depth+1))]

    def process_path(path: list[int]) -> tuple[float, list[int]]:
        visited = [False] * N
        visited[0] = True
        curr_bound = 0
        curr_path = [-1] * (N + 1)
        curr_path[0] = path[0]
        for i in range(N):
            curr_bound += (first_min(dist_mat, i) + second_min(dist_mat, i))
            curr_bound = math.ceil(curr_bound / 2)

        final_res = [MAXSIZE]
        final_path = [-1] * (N + 1)

```

```

    tsp_recursive(
        dist_mat,
        curr_bound,
        0,
        1,
        curr_path,
        visited,
        final_path,
        N,
        final_res
    )
    return final_res[0], final_path

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    initial_paths = comm.bcast(initial_paths, root=0)

    results = []
    for i, path in enumerate(initial_paths):
        result = process_path(path)
        results.append(result)

    final_res = [MAXSIZE]
    final_path = [-1] * (N + 1)
    for res in results:
        if res[0] < final_res[0]:
            final_res[0] = res[0]
            final_path = res[1]

    return final_res[0], final_path

return None, None

```

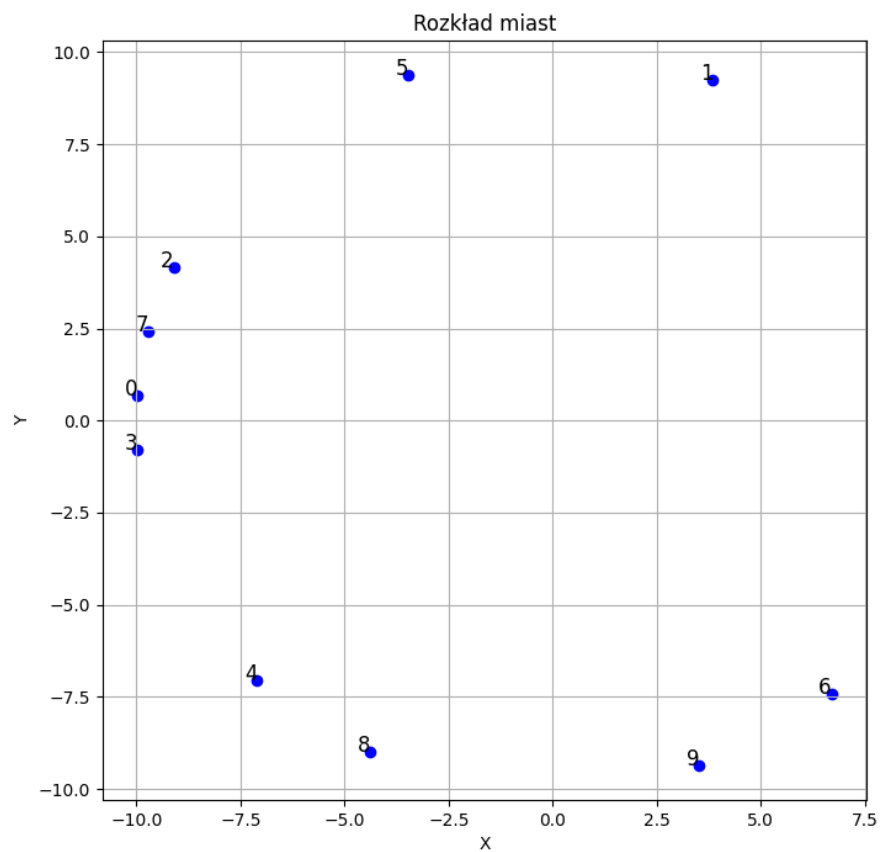
4 Eksperymenty

W ramach eksperymentów rozważymy problemu rozmiaru $N = 10$ i $N = 15$. Sprawdzona będzie również generacja na i wewnątrz okręgu. Parametry odpowiedzialne za losowość rozłożenia miast (szum) będą stałe dla wszystkich eksperymentów.

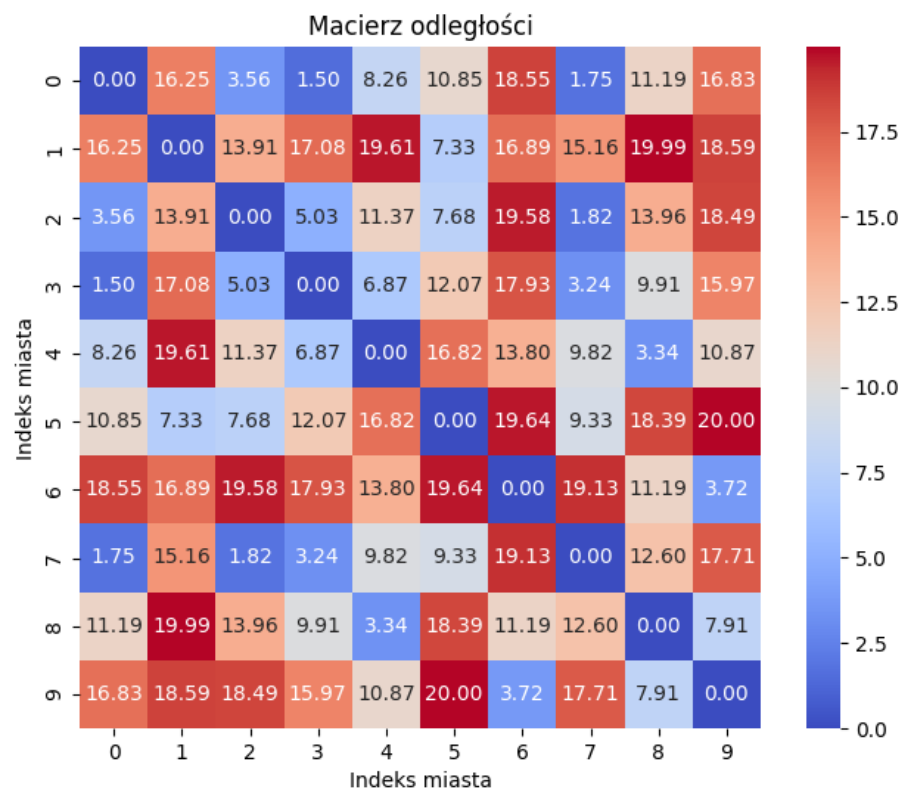
4.1 Implementacja sekwencyjna

4.1.1 Rozmiar $N = 10$

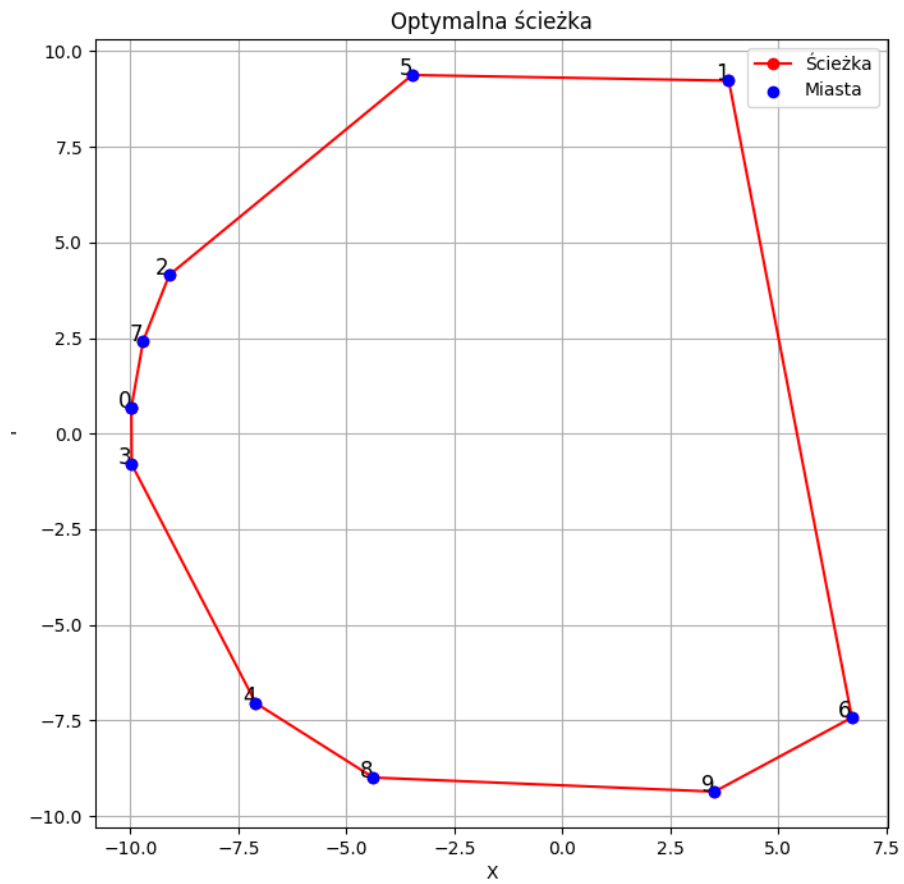
Miasta na okregu :



Rysunek 1: Rozkład miast



Rysunek 2: Heatmapa odległości między miastami



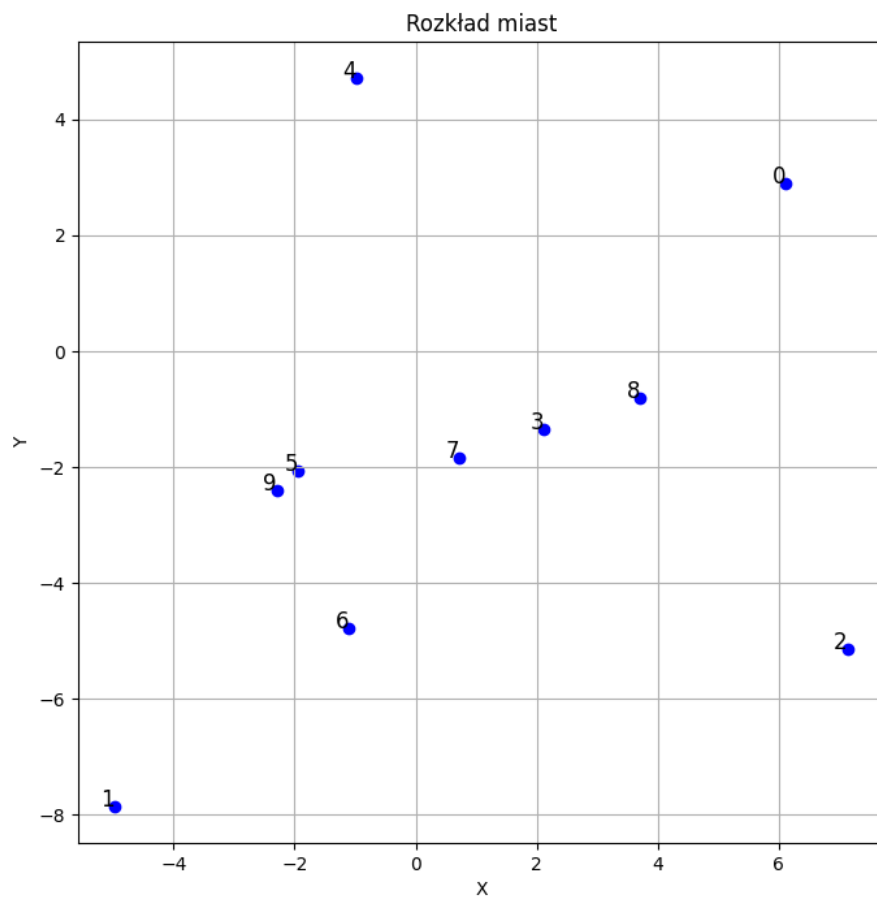
Rysunek 3: Optymalna ścieżka

Algorytm wybrał ścieżkę:

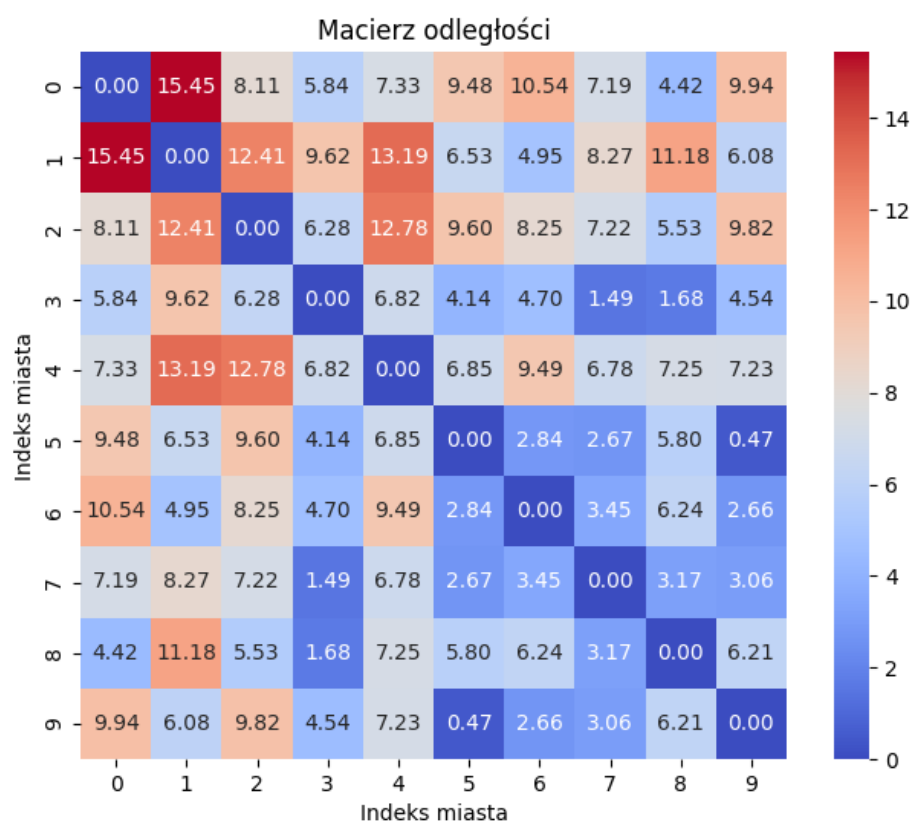
$$0 \rightarrow 3 \rightarrow 4 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow 1 \rightarrow 5 \rightarrow 2 \rightarrow 7 \rightarrow 0$$

jej całkowity koszt to **58.8202475232184**. Algorytm wykonał się w **0.07** sekundy.

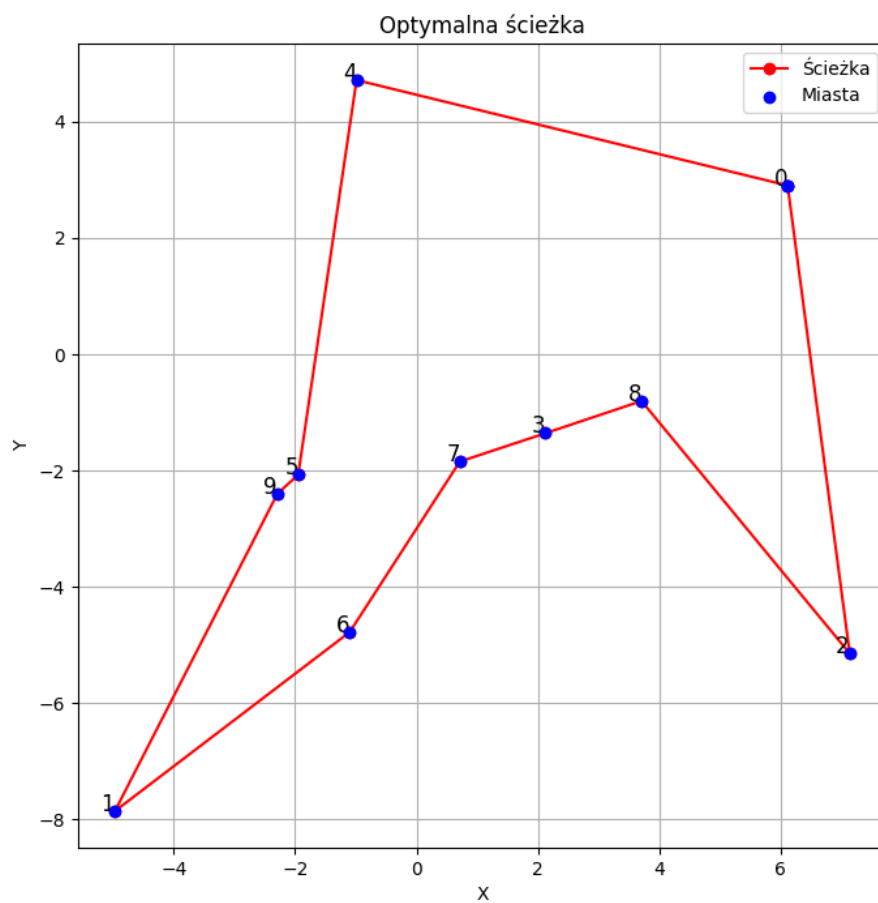
Miasta wewnątrz okręgu :



Rysunek 4: Rozkład miast wewnątrz okręgu



Rysunek 5: Heatmapa odległości miast.



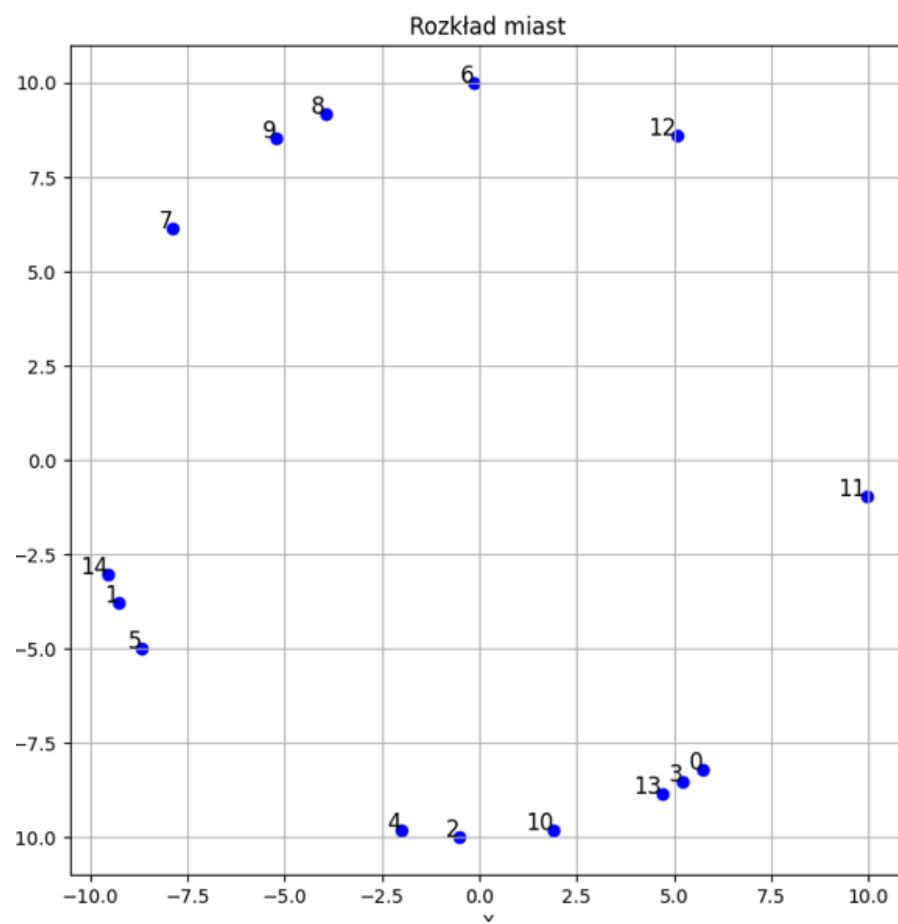
Rysunek 6: Optymalna ścieżka

Algorytm wybrał ścieżkę:

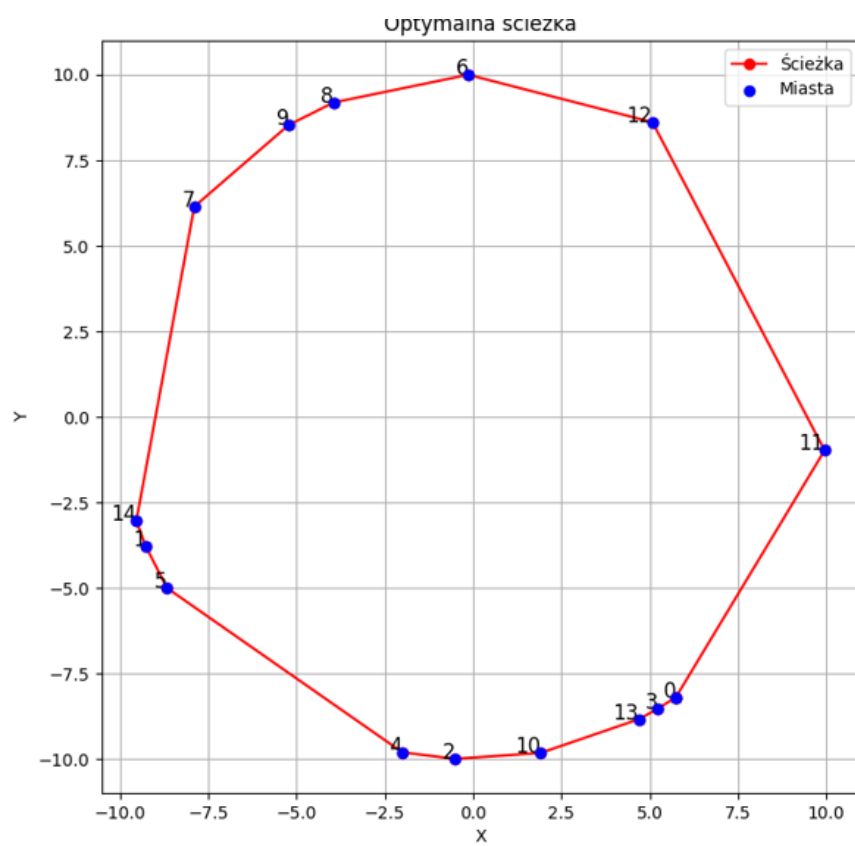
$$0 \rightarrow 2 \rightarrow 8 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 1 \rightarrow 9 \rightarrow 5 \rightarrow 4 \rightarrow 0$$

jej całkowity koszt to **45.94564798204372**. Algorytm wykonał się w **0.28** sekundy.

4.1.2 Rozmiar $N = 15$



Rysunek 7: Rozkład miast na okręgu

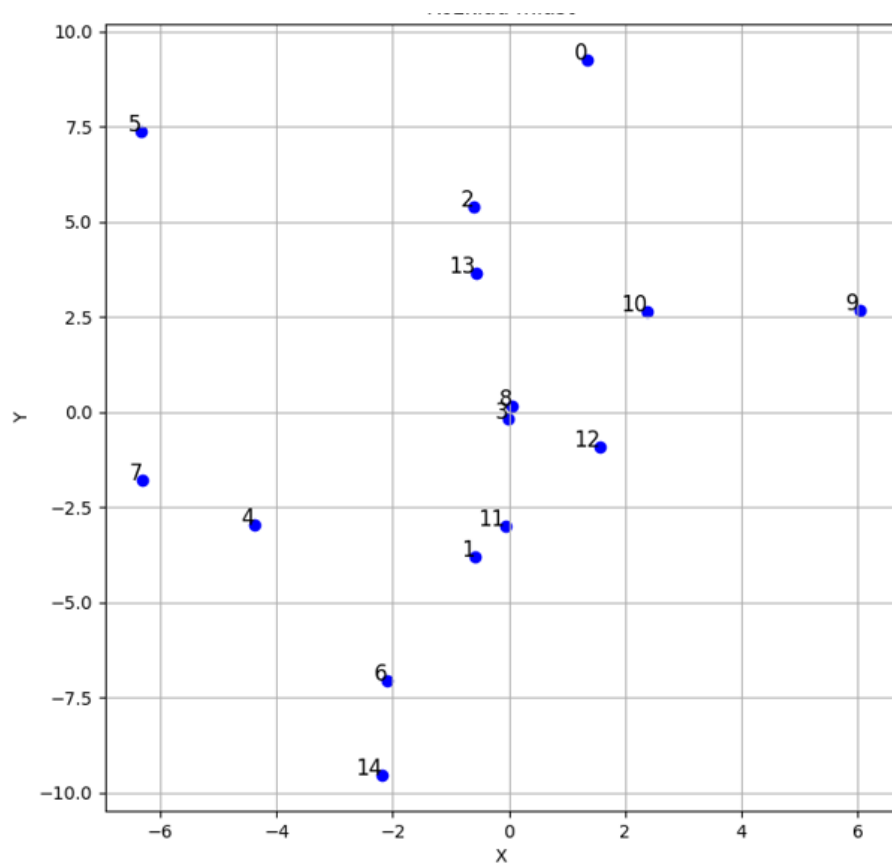


Rysunek 8: Optymalna ścieżka

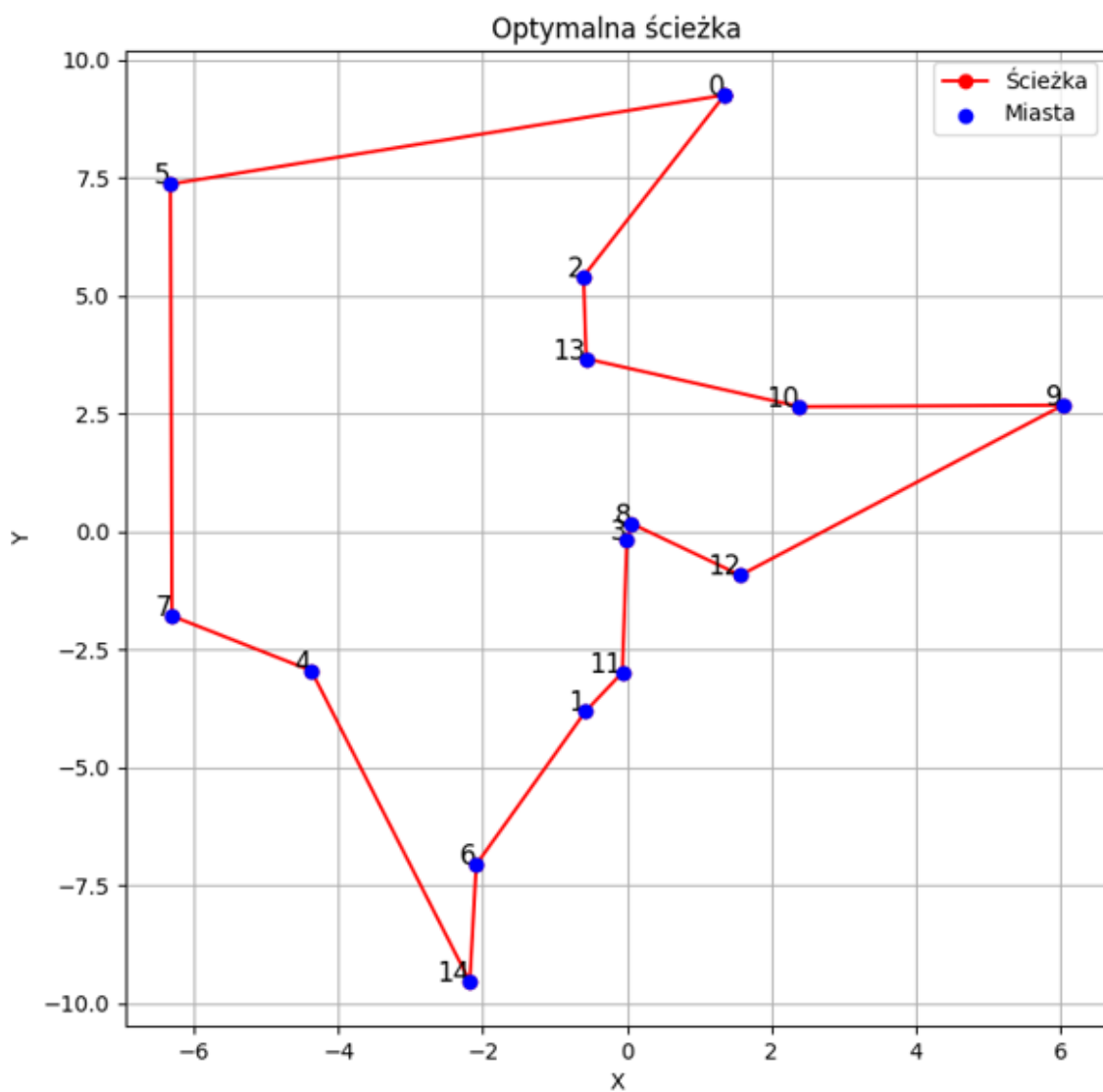
Miasta na okręgu Algorytm wybrał ścieżkę:

$0 \rightarrow 3 \rightarrow 13 \rightarrow 10 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 14 \rightarrow 7 \rightarrow 9 \rightarrow 8 \rightarrow 6 \rightarrow 12 \rightarrow 11 \rightarrow 0$

jej całkowity koszt to **45.94564798204372**. Algorytm wykonał się w **9.43** sekundy.



Rysunek 9: Rozkład miast wewnątrz okręgu



Rysunek 10: Optymalna ścieżka

Miasta wewnątrz okręgu Algorytm wybrał ścieżkę:

$0 \rightarrow 2 \rightarrow 13 \rightarrow 10 \rightarrow 9 \rightarrow 12 \rightarrow 8 \rightarrow 3 \rightarrow 11 \rightarrow 1 \rightarrow 6 \rightarrow 14 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 0$

jej całkowity koszt to **56.89942607195136**. Algorytm wykonał się w **94.98** sekundy.

Wnioski : Widzimy, że rozmiar problemu i rozłożenie miast mają bardzo duży wpływ na szybkość wykonania algorytmu.

4.2 Implementacja równoległa

Eksperymenty dla wersji równoległej przeprowadzimy dla problemu o rozmiarze $N = 15$ oraz dla miast wygenerowanych **wewnątrz** okręgu przy zmiennym parametrze **d** i liczbie procesów.

d = 2, 2 procesy Execution time: 321.55s

Minimum cost: 61.06028603428557

Path taken: [0, 4, 1, 7, 3, 10, 6, 11, 14, 9, 8, 13, 5, 12, 2, 0]

Top Hotspots Function	Module	CPU Time	% of CPU Time(%)
PyEval_EvalFrameDefault	libpython3.9.so.1.0	24.499s	14.2%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	13.710s	7.9%
pymalloc_alloc	libpython3.9.so.1.0	7.740s	4.5%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	6.830s	3.9%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	6.760s	3.9%
[Others]	N/A	113.441s	65.6%
Effective CPU Utilization: 2.1%			
The metric value is low, which may signal a poor logical CPU cores utilization caused by load imbalance, threading runtime overhead, contended synchronization, or thread/process underutilization. Explore sub-metrics to estimate the efficiency of MPI and OpenMP parallelism or run the Locks and Waits analysis to identify parallel bottlenecks for other parallel runtimes.			
Average Effective CPU Utilization: 0.984 out of 48			

d = 3, 2 procesy Execution time: 20.03s

Minimum cost: 48.60273717808537

Path taken: [0, 9, 6, 8, 3, 7, 10, 14, 1, 4, 2, 11, 13, 5, 12, 0]

Top Hotspots Function	Module	CPU Time	% of CPU Time(%)
PyEval_EvalFrameDefault	libpython3.9.so.1.0	53.890s	14.6%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	27.830s	7.5%
pymalloc_alloc	libpython3.9.so.1.0	15.800s	4.3%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	15.350s	4.2%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	14.170s	3.8%
[Others]	N/A	241.970s	65.6%
Effective CPU Utilization: 2.1%			
The metric value is low, which may signal a poor logical CPU cores utilization caused by load imbalance, threading runtime overhead, contended synchronization, or thread/process underutilization. Explore sub-metrics to estimate the efficiency of MPI and OpenMP parallelism or run the Locks and Waits analysis to identify parallel bottlenecks for other parallel runtimes.			
Average Effective CPU Utilization: 0.990 out of 48			

d = 4, 2 procesy Execution time: 41.67 Minimum cost: 52.608882472531604 Path taken: [0, 1, 10, 5, 6, 12, 9, 2, 8, 11, 4, 7, 14, 13, 3, 0]

CPU Time: 63.140s			
Effective Time: 63.140s			
Spin Time: 0s			
Overhead Time: 0s			
Total Thread Count: 2			
Paused Time: 0s			
Top Hotspots			
Function	Module	CPU Time	% of CPU Time(%)

_PyEval_EvalFrameDefault	libpython3.9.so.1.0	8.600s	13.6%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	4.150s	6.6%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	2.710s	4.3%
pymalloc_alloc	libpython3.9.so.1.0	2.560s	4.1%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	2.340s	3.7%
[Others]	N/A	42.780s	67.8%
Effective CPU Utilization: 2.0%			

Rysunek 11: Enter Caption

d = 2, 4 procesy Execution time: 166.67 Minimum cost: 57.183862845001066 Path taken: [0, 3, 5, 7, 10, 4, 13, 9, 14, 6, 11, 1, 8, 12, 2, 0]

Top Hotspots			
Function	Module	CPU Time	% of CPU Time(%)

_PyEval_EvalFrameDefault	libpython3.9.so.1.0	15.060s	12.2%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	8.440s	6.8%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	5.260s	4.2%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	4.720s	3.8%
pymalloc_alloc	libpython3.9.so.1.0	4.600s	3.7%
[Others]	N/A	85.820s	69.3%
Effective CPU Utilization: 2.0%			

d = 3, 4 procesy Execution time: 42.71 Minimum cost: 44.04535449414117 Path taken: [0, 3, 9, 4, 5, 2, 7, 10, 6, 13, 8, 1, 14, 11, 12, 0]

CPU Time: 1548.720s			
Effective Time: 1548.720s			
Spin Time: 0s			
Overhead Time: 0s			
Total Thread Count: 2			
Paused Time: 0s			
Top Hotspots			
Function	Module	CPU Time	% of CPU Time(%)

_PyEval_EvalFrameDefault	libpython3.9.so.1.0	184.010s	11.9%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	95.220s	6.1%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	68.750s	4.4%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	60.520s	3.9%
pymalloc_alloc	libpython3.9.so.1.0	57.530s	3.7%
[Others]	N/A	1082.690s	69.9%
Effective CPU Utilization: 2.1%			

d = 4, 4 procesy Execution time: 89.96 Minimum cost: 60.22844999730788 Path taken: [0, 1, 13, 7, 5, 2, 3, 4, 12, 9, 14, 10, 6, 8, 11, 0]

CPU Time: 259.810s			
Effective Time: 259.810s			
Spin Time: 0s			
Overhead Time: 0s			
Total Thread Count: 2			
Paused Time: 0s			
Top Hotspots			
Function	Module	CPU Time	% of CPU Time(%)
PyEval_EvalFrameDefault	libpython3.9.so.1.0	30.530s	11.8%
prepare_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	16.400s	6.3%
PyArray_NewFromDescr_int	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	12.740s	4.9%
get_view_from_index	_multiarray_umath.cpython-39-x86_64-linux-gnu.so	10.060s	3.9%
_memset_avx2_unaligned_erms	libc.so.6	9.760s	3.8%
[Others]	N/A	180.320s	69.4%
Effective CPU Utilization: 2.1%			

4.3 Przyspieszenie, efektywność, metryka Karpa-Flatta

